Scoped Chat Participants in Visual Studio Code

Question: A developer is working in Visual Studio Code and encounters a failing shell script execution inside the integrated terminal. They open the GitHub Copilot Chat view and want Copilot to inspect the recent terminal command output and suggest a corrected CLI command. Which chat participant should they prefix their prompt with?

A. `@workspace` — Searches across all files and directories in the open repository.

B. `@terminal` — Scopes the prompt to the integrated terminal's shell environment and recent command history. **[CORRECT ANSWER]**

C. `@vscode` — Answers questions exclusively about Visual Studio Code settings, keybindings, and editor commands.

D. `#terminalSelection` — Attaches the entire repository build log automatically without user selection.

Explanation & Exam Rationale

Chat participants (invoked with the `@` symbol) act as domain-specialized agents inside GitHub Copilot Chat that gather targeted environmental context before answering a prompt. Using `@terminal` allows Copilot to read the active shell type, recent command exit codes, and terminal output to diagnose shell errors or synthesize valid terminal commands.

Participant / Variable	Primary Context Scope	Typical Exam Use Case
`@workspace`	Entire open project structure & indexed files	Cross-file refactoring, locating symbols, architectural questions
`@terminal`	Integrated terminal shell & recent command output	Fixing failed CLI commands, generating shell/git pipelines
`@vscode`	VS Code editor UI, settings, extensions & commands	Configuring `settings.json`, finding editor shortcuts
`#terminalLastCommand`	Context variable for the most recent terminal command	Explicitly injecting the last executed command and its output

Key Exam Takeaway: Distinguish between `@` *participants* (specialized agents that route your request) and `#` *variables* (explicit context attachments such as `#file` or `#selection`).